

Plan d'implémentation du formateur AutoLISP

OpenAI

April 27, 2026

Contents

1	Principes d'organisation	3
2	Arborescence suggérée	3
3	Lots d'implémentation	4
3.1	LOT 0 — cadrage et conventions	4
3.1.1	Tâches	4
3.1.2	Livrables	4
3.1.3	Critère de fin	4
3.2	LOT 1 — infrastructure de base	5
3.2.1	Tâches	5
3.2.2	Tests	5
3.2.3	Critère de fin	5
3.3	LOT 2 — scanner / tokenizer	5
3.3.1	Objectif	5
3.3.2	Tâches	5
3.3.3	Jeux de tests	6
3.3.4	Critère de fin	6
3.4	LOT 3 — parseur léger / CST	6
3.4.1	Objectif	6
3.4.2	Tâches	6
3.4.3	Jeux de tests	7
3.4.4	Critère de fin	7
3.5	LOT 4 — pretty-printer minimal	7
3.5.1	Objectif	7
3.5.2	Tâches	7
3.5.3	Jeux de tests	8
3.5.4	Critère de fin	8

3.6	LOT 5 — styles de parenthèses	8
3.6.1	Objectif	8
3.6.2	Tâches	8
3.6.3	Jeux de tests	8
3.6.4	Critère de fin	9
3.7	LOT 6 — règles d’indentation spécifiques	9
3.7.1	Objectif	9
3.7.2	Tâches	9
3.7.3	Jeux de tests	9
3.7.4	Critère de fin	9
3.8	LOT 7 — normalisation de casse des symboles	10
3.8.1	Objectif	10
3.8.2	Tâches	10
3.8.3	Jeux de tests	10
3.8.4	Critère de fin	10
3.9	LOT 8 — politique de commentaires	10
3.9.1	Objectif	10
3.9.2	Tâches	10
3.9.3	Jeux de tests	11
3.9.4	Critère de fin	11
3.10	LOT 9 — interface utilisateur et mode check	11
3.10.1	Objectif	11
3.10.2	Tâches	11
3.10.3	Jeux de tests	12
3.10.4	Critère de fin	12
3.11	LOT 10 — corpus réel et non-régression	12
3.11.1	Objectif	12
3.11.2	Tâches	12
3.11.3	Jeux de tests	12
3.11.4	Critère de fin	13
4	Tests détaillés à prévoir	13
4.1	Scanner	13
4.2	Parseur	13
4.3	Printer	14
4.4	Transformations	14
4.5	Intégration	14
5	Dépendances entre lots	14

6 Jalons suggérés	15
6.1 Jalon A — noyau syntaxique	15
6.2 Jalon B — formatage minimal exploitable	15
6.3 Jalon C — lisibilité équipe	15
6.4 Jalon D — industrialisation	15
7 Critères d’achèvement globaux	15

1 Principes d’organisation

Le travail doit être découpé en modules indépendants autant que possible, avec une progression incrémentale :

1. obtenir un scanner fiable ;
2. construire un parseur syntaxique léger ;
3. produire un pretty-printer minimal ;
4. ajouter les options de style ;
5. sécuriser par tests ;
6. valider sur corpus réel.

Le projet doit viser une première version utile rapidement, puis durcir la couverture et les cas limites.

2 Arborescence suggérée

```
src/
  fmt-main.lsp
  fmt-options.lsp
  fmt-io.lsp
  fmt-scanner.lsp
  fmt-parser.lsp
  fmt-cst.lsp
  fmt-printer.lsp
  fmt-rules.lsp
  fmt-comments.lsp
  fmt-case.lsp
```

```
fmt-errors.lsp
fmt-tests.lsp
fmt-golden.lsp
fmt-util.lsp
tests/
  scanner/
  parser/
  printer/
  golden/
  corpus/
docs/
  specifications.org
  plan.org
```

3 Lots d'implémentation

3.1 LOT 0 — cadrage et conventions

3.1.1 Tâches

- figer la spécification des options ;
- figer les noms de modules ;
- définir la représentation des tokens ;
- définir la représentation des nœuds CST ;
- choisir la stratégie de rendu des erreurs ;
- définir les conventions de tests.

3.1.2 Livrables

- conventions internes ;
- schéma des structures de données ;
- jeu d'exemples minimaux.

3.1.3 Critère de fin

Toute l'équipe peut lire et discuter les structures avant codage lourd.

3.2 LOT 1 — infrastructure de base

3.2.1 Tâches

- module utilitaire (fonctions listes, chaînes, positions) ;
- module I/O (lecture/écriture fichiers) ;
- structure de résultat / erreur ;
- infrastructure de tests simple ;
- fonction de comparaison de sorties exactes.

3.2.2 Tests

- lecture d'un fichier texte simple ;
- écriture d'un fichier ;
- remontée d'erreur standardisée.

3.2.3 Critère de fin

Le squelette du projet s'exécute et permet d'ajouter les modules suivants proprement.

3.3 LOT 2 — scanner / tokenizer

3.3.1 Objectif

Transformer le texte source en tokens fiables, avec gestion correcte des commentaires et chaînes.

3.3.2 Tâches

- implémenter un curseur de lecture caractère par caractère ;
- reconnaître espace, tab, newline ;
- reconnaître (et) ;
- reconnaître quote ;
- reconnaître chaînes ;
- reconnaître commentaires de ligne ;

- reconnaître commentaires bloc `;| ... |;` ;
- reconnaître atomes ;
- stocker ligne/colonne ou offset ;
- produire une API `scan-string` et `scan-file`.

3.3.3 Jeux de tests

- tokens sur expressions simples ;
- parenthèses dans chaînes ;
- parenthèses dans commentaires ;
- bloc `;| ... |;` multi-ligne ;
- symboles `:foo`, `pk!`, `vl-catch-all-apply` ;
- dotted pair.

3.3.4 Critère de fin

Le scanner passe tous les tests unitaires et produit des tokens utilisables par le parseur.

3.4 LOT 3 — parseur léger / CST

3.4.1 Objectif

Reconstruire la structure des s-expressions et attacher les trivia sans évaluation.

3.4.2 Tâches

- définir nœuds atome, chaîne, liste, quote, trivia ;
- parser les formes top-level ;
- parser les listes imbriquées ;
- gérer les quotes ;
- attacher trivia préfixe/suffixe ;

- détecter parenthèses non équilibrées ;
- détecter fin de chaîne ou commentaire bloc non terminé.

3.4.3 Jeux de tests

- forme simple ;
- plusieurs formes top-level ;
- commentaires avant/après forme ;
- quote simple et quote de liste ;
- erreur de parenthèse ;
- erreur de commentaire bloc non fermé.

3.4.4 Critère de fin

Le parseur produit un CST suffisamment précis pour permettre le pretty-printing.

3.5 LOT 4 — pretty-printer minimal

3.5.1 Objectif

Produire un rendu stable sans encore traiter toutes les subtilités de style.

3.5.2 Tâches

- moteur d'indentation générique ;
- émission des atomes ;
- émission des chaînes ;
- émission des listes ;
- rendu top-level multi-formes ;
- stratégie simple pour commentaires préservés ;
- normalisation minimale des lignes vides.

3.5.3 Jeux de tests

- liste plate ;
- liste imbriquée ;
- fichier multi-formes ;
- commentaires préservés ;
- test d'idempotence simple.

3.5.4 Critère de fin

Un corpus simple peut déjà être reformatté de façon stable.

3.6 LOT 5 — styles de parenthèses

3.6.1 Objectif

Ajouter les modes `cl` et `nail`.

3.6.2 Tâches

- isoler la politique d'émission des fermetures ;
- implémenter style `cl` ;
- implémenter style `nail` ;
- ajouter l'option utilisateur correspondante ;
- golden tests comparatifs.

3.6.3 Jeux de tests

- mêmes entrées, sorties attendues différentes selon style ;
- `if/progn` imbriqués ;
- `defun` avec code interne multi-niveau ;
- style `nail` sur code déjà `nail` ;
- conversion `nail` -> `cl` -> `nail` stable.

3.6.4 Critère de fin

Le passage d'un style à l'autre est fiable et déterministe.

3.7 LOT 6 — règles d'indentation spécifiques

3.7.1 Objectif

Améliorer la lisibilité des formes courantes AutoLISP.

3.7.2 Tâches

- définir table de règles par tête de forme ;
- implémenter règles pour `defun` ;
- `lambda` ;
- `if` ;
- `cond` ;
- `setq` ;
- `progn` ;
- `while` ;
- `repeat` ;
- `foreach` ;
- prévoir extensibilité future.

3.7.3 Jeux de tests

- un golden test par forme ;
- formes imbriquées ;
- listes d'arguments avec `/` pour variables locales.

3.7.4 Critère de fin

Le rendu est lisible sur des fonctions réelles d'équipe.

3.8 LOT 7 — normalisation de casse des symboles

3.8.1 Objectif

Ajouter les modes `preserve`, `downcase`, `upcase`.

3.8.2 Tâches

- identifier précisément les tokens transformables ;
- ne jamais modifier chaînes ni commentaires ;
- implémenter transformation des symboles ;
- intégrer au pipeline d'émission.

3.8.3 Jeux de tests

- symboles simples ;
- symboles avec `:` ;
- symboles avec `!`, `*`, `-` ;
- vérification de non-modification des chaînes ;
- vérification de non-modification des commentaires.

3.8.4 Critère de fin

Les trois modes donnent les sorties attendues sans altérer le texte non symbolique.

3.9 LOT 8 — politique de commentaires

3.9.1 Objectif

Supporter `preserve` et normalisation inspirée CL.

3.9.2 Tâches

- classer les commentaires selon contexte ;
- mode `preserve` ;
- mode `cl` ;

- conversion ;| ... |; -> lignes préfixées ;
- option contextuelle ;
- alignement des commentaires de fin de ligne.

3.9.3 Jeux de tests

- commentaire top-level ;
- commentaire de section ;
- commentaire dans sexp ;
- commentaire de fin de ligne ;
- conversion d'un bloc multi-ligne ;
- alignement sur colonne.

3.9.4 Critère de fin

Le rendu des commentaires est stable et conforme aux options choisies.

3.10 LOT 9 — interface utilisateur et mode check

3.10.1 Objectif

Rendre l'outil utilisable par l'équipe et la CI.

3.10.2 Tâches

- parser les options ;
- support `--check` ;
- support `--in-place` ;
- support sortie vers fichier ;
- codes retour cohérents ;
- messages d'erreur lisibles.

3.10.3 Jeux de tests

- check fichier conforme ;
- check fichier non conforme ;
- formatage in place ;
- sortie vers fichier ;
- combinaison d'options.

3.10.4 Critère de fin

L'outil est utilisable dans des scripts et pipelines d'équipe.

3.11 LOT 10 — corpus réel et non-régression

3.11.1 Objectif

Valider sur code AutoLISP existant et figer des golden tests réalistes.

3.11.2 Tâches

- constituer un corpus représentatif ;
- sélectionner fichiers simples, moyens, difficiles ;
- exécuter le formateur ;
- inspecter les écarts ;
- corriger règles ;
- figer sorties de référence.

3.11.3 Jeux de tests

- golden corpus complet ;
- double passage = idempotence ;
- validation manuelle d'un échantillon.

3.11.4 Critère de fin

L'équipe peut lancer l'outil sur son code sans corruption et avec confiance raisonnable.

4 Tests détaillés à prévoir

4.1 Scanner

- expression minimale ;
- espaces et tabulations ;
- fins de ligne CR/LF si nécessaire ;
- commentaire ligne ;
- commentaire bloc ;
- chaîne contenant ; ;
- chaîne contenant parenthèses ;
- symboles exotiques ;
- dotted pairs.

4.2 Parseur

- listes simples ;
- formes top-level multiples ;
- quote ;
- commentaire entre deux formes ;
- erreur EOF dans liste ;
- erreur EOF dans chaîne ;
- erreur EOF dans commentaire bloc.

4.3 Printer

- indentation de base ;
- rendu top-level ;
- rendu des commentaires ;
- style `cl` ;
- style `nail` ;
- stabilité des lignes vides.

4.4 Transformations

- preserve/downcase/upcase ;
- commentaires preserve/cl ;
- conversion bloc -> semicolons ;
- inline comments column.

4.5 Intégration

- scan + parse + print sur petit corpus ;
- check mode ;
- in place ;
- idempotence ;
- non-régression.

5 Dépendances entre lots

- LOT 0 précède tout ;
- LOT 1 précède LOT 2 à 10 ;
- LOT 2 précède LOT 3 ;
- LOT 3 précède LOT 4 ;

- LOT 4 précède LOT 5 à 8 ;
- LOT 5, 6, 7, 8 peuvent partiellement avancer en parallèle une fois LOT 4 stabilisé ;
- LOT 9 dépend des options déjà stabilisées ;
- LOT 10 est continu mais se fige en fin de projet.

6 Jalons suggérés

6.1 Jalon A — noyau syntaxique

Inclut LOT 0 à 3.

Résultat : scanner + parseur utilisables.

6.2 Jalon B — formatage minimal exploitable

Inclut LOT 4 et LOT 5.

Résultat : le projet sait déjà reformater les parenthèses dans les deux styles.

6.3 Jalon C — lisibilité équipe

Inclut LOT 6, 7, 8.

Résultat : formatage réellement utile au quotidien.

6.4 Jalon D — industrialisation

Inclut LOT 9 et LOT 10.

Résultat : intégration outillage / CI / corpus réel.

7 Critères d'achèvement globaux

Le projet est terminé lorsque :

- tous les lots critiques sont implémentés ;
- les golden tests sont stables ;
- l'idempotence est démontrée ;
- un corpus réel d'équipe passe correctement ;

- les options documentées sont supportées ;
- l'outil est suffisamment robuste pour être adopté en pré-commit ou en revue de code.